

Open Weather Flutter

Material de estudo e apresentação

Aplicativo Flutter de clima com API OpenWeather.

Inclui busca de cidades, histórico, alertas e mapa climático interativo.

Projeto organizado em camadas com Riverpod, Dio, Geolocator e Flutter Map.

Tema: Aplicativo mobile de previsão do tempo

Linguagem: Dart

Framework: Flutter

API: OpenWeather

Objetivo: Estudo, demonstração e apresentação acadêmica

Material de estudo e apresentação do projeto

Aluno: Wagner

Curso: Projeto Flutter com API OpenWeather

1. Visão geral

O Open Weather Flutter é um aplicativo mobile desenvolvido em Flutter para consultar informações meteorológicas usando a API da OpenWeather.

O objetivo do projeto é demonstrar uma aplicação real com consumo de API, gerenciamento de estado, navegação, geolocalização, busca de cidades, histórico, alertas e mapa climático interativo.

De forma prática, o usuário consegue abrir o app, permitir o uso da localização ou pesquisar uma cidade, visualizar a previsão atual, consultar alertas meteorológicos, revisar as últimas cidades buscadas e abrir um mapa com camadas de chuva, nuvens, temperatura e vento.

2. Problema resolvido

Muitos aplicativos de clima mostram apenas a temperatura atual. Este projeto busca ir além, reunindo em uma interface simples:

- clima atual;
- busca de cidade;
- alertas;
- histórico de consultas;
- mapa climático com camadas visuais.

Assim, o usuário não fica limitado a um único número de temperatura. Ele consegue entender melhor a condição meteorológica da região.

3. Principais funcionalidades

- Tela inicial com clima atual.
- Busca de cidade usando geocoding da OpenWeather.
- Atualização do clima conforme a cidade selecionada.
- Histórico das 5 últimas cidades buscadas.
- Tela de alertas meteorológicos.
- Mapa interativo com camadas OpenWeather.

- Menu lateral para navegação.
- Localização atual via GPS.
- Interface em português do Brasil.

4. Tecnologias utilizadas

O projeto utiliza Flutter e Dart como base principal.

Também foram usadas bibliotecas importantes:

- flutter_riverpod: gerenciamento de estado.
- dio: requisições HTTP.
- geolocator: acesso à localização do usuário.
- flutter_dotenv: leitura da chave da API em arquivo .env.
- flutter_map: mapa interativo leve.
- latlong2: representação de coordenadas geográficas.
- intl: formatação de datas em português.

5. Estrutura de pastas

O projeto foi organizado em camadas para separar responsabilidades.

```
lib/  
  core/  
    constants/  
    errors/  
    helpers/  
    navigation/  
  data/  
    models/  
    repositories/  
  domain/  
    providers/  
  presentation/  
    screens/  
    widgets/
```

Essa separação facilita manutenção, testes e evolução do app.

6. Papel de cada camada

core: contém elementos reutilizáveis e centrais, como constantes, navegação, helpers e tratamento de erros.

data: contém modelos e repositórios. É a camada responsável por conversar com APIs externas e transformar JSON em objetos Dart.

domain: contém providers e regras de estado da aplicação, como clima selecionado, histórico e configurações.

presentation: contém telas e widgets. É a camada visual do aplicativo.

7. Inicialização do app

O arquivo main.dart é o ponto de entrada do projeto.

Ele faz três coisas principais:

1. Inicializa os bindings do Flutter.
2. Carrega o arquivo .env com a chave da OpenWeather.
3. Inicia o app com ProviderScope, necessário para usar Riverpod.

Trecho conceitual:

```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await dotenv.load(fileName: '.env');  
  runApp(const ProviderScope(child: MyApp()));  
}
```

8. Chave da API

A chave da OpenWeather não fica escrita diretamente no código.

Ela é lida a partir do arquivo .env:

```
OPENWEATHER_API_KEY=SUA_CHAVE_AQUI
```

Isso evita expor credenciais no código-fonte.

No projeto, a classe ApiConstants centraliza as URLs da API:

```
static String get apiKey =>  
  (dotenv.env['OPENWEATHER_API_KEY'] ?? '').trim();
```

9. Consumo da API OpenWeather

O app usa duas partes importantes da OpenWeather:

1. One Call API: retorna clima atual, previsão, alertas e dados por coordenada.
2. Geocoding API: transforma texto digitado pelo usuário em cidades com latitude e longitude.

O fluxo é:

```
Usuário pesquisa cidade
-> app consulta Geocoding API
-> usuário escolhe uma sugestão
-> app salva latitude e longitude
-> app consulta One Call API
-> tela atualiza com clima da cidade
```

10. Modelo de dados

O arquivo `weather_model.dart` define as classes que representam os dados meteorológicos.

Principais modelos:

- WeatherModel
- CurrentWeather
- DailyWeather
- HourlyWeather
- WeatherAlert

Esses modelos convertem o JSON da OpenWeather em objetos Dart seguros e fáceis de usar na interface.

Exemplo de responsabilidade:

```
JSON da API -> WeatherModel -> widgets da tela
```

11. Repositório de clima

O WeatherRepository é responsável pelas chamadas HTTP.

Ele usa Dio para buscar:

- clima por latitude/longitude;
- sugestões de cidades pelo nome.

Essa separação é importante porque a tela não precisa saber detalhes de URL, query parameters ou tratamento HTTP.

12. Gerenciamento de estado com Riverpod

Riverpod foi usado para controlar dados compartilhados entre telas.

Providers importantes:

- `weatherProvider`: busca e entrega o clima atual.
- `selectedLocationProvider`: guarda latitude e longitude da cidade selecionada.
- `selectedCityNameProvider`: guarda o nome da cidade selecionada.
- `historyProvider`: guarda as últimas cidades buscadas.

Com isso, quando uma cidade é selecionada, as telas que dependem do clima são atualizadas automaticamente.

13. Tela Home

A Home é a tela principal do app.

Ela mostra:

- horário atual;
- data;
- cidade;
- temperatura;
- descrição do clima;
- sensação térmica;
- umidade;
- vento;
- previsão por hora;
- previsão semanal;
- campo de busca de cidades.

Ela também é a origem do histórico: cada cidade buscada na Home pode entrar na lista das últimas cidades consultadas.

14. Busca de cidade

O campo de busca usa um `debounce`, ou seja, espera um pequeno intervalo antes de chamar a API.

Isso evita chamadas excessivas enquanto o usuário ainda está digitando.

Fluxo simplificado:

```
Usuário digita
-> app espera alguns milissegundos
-> chama API de geocoding
-> mostra sugestões
-> usuário escolhe uma cidade
-> estado global é atualizado
```

15. Histórico

A tela Histórico substituiu a antiga ideia de Favoritos.

Ela guarda as 5 últimas cidades buscadas.

Regras:

- a cidade mais recente fica no topo;
- cidades duplicadas não aparecem repetidas;
- se uma cidade já existe no histórico, ela sobe para o topo;
- ao tocar em uma cidade, a Home carrega novamente o clima daquela cidade.

O histórico atual fica em memória. Isso significa que ele reinicia quando o app é fechado.

16. Tela de alertas

A tela de alertas mostra avisos meteorológicos retornados pela OpenWeather.

Quando não existem alertas ativos, ela mostra uma mensagem amigável:

```
Nenhum alerta ativo
```

Também são exibidas dicas do dia com base nas condições atuais, como umidade elevada, vento forte ou céu aberto.

17. Mapa climático

A tela Mapa usa `flutter_map` para exibir um mapa interativo.

Ela combina:

- mapa base do OpenStreetMap;
- camada climática da OpenWeather por cima.

As camadas disponíveis são:

- Chuva;
- Nuvens;
- Temperatura;
- Vento.

Cada camada é uma URL de tiles:

```
https://tile.openweathermap.org/map/{layer}/{z}/{x}/{y}.png?appid=SUA_CHAVE
```

18. Por que flutter_map?

O flutter_map foi escolhido por ser uma solução leve.

Ele evita dependências pesadas como SDKs nativos de mapas.

Vantagens:

- funciona com tiles raster;
- é simples de configurar;
- permite sobrepor camadas;
- é suficiente para o objetivo do projeto;
- combina bem com as tiles da OpenWeather.

19. Controles do mapa

O mapa possui:

- arrastar para mover;
- gesto de pinça para aproximar;
- duplo toque para zoom;
- botões + e - para facilitar o uso;
- botão para centralizar na cidade atual;
- seleção de camada climática.

Isso melhora a usabilidade principalmente em emuladores e celulares.

20. Menu lateral

O menu lateral permite navegar entre as telas:

- Início;
- Mapa;
- Histórico;
- Alertas;
- Configurações;
- Sobre.

Ele é controlado por uma GlobalKey específica do EdgeMenu.

21. Navegação

A navegação é centralizada no AppRouter.

Rotas principais:

```
/          -> Home
/map       -> Mapa
/history   -> Histórico
/alerts    -> Alertas
/settings  -> Configurações
/about     -> Sobre
```

Centralizar rotas facilita manutenção e evita strings espalhadas pelo projeto.

22. Permissões

O app usa localização do dispositivo.

Ao iniciar sem cidade selecionada, ele solicita permissão de localização.

Se a permissão for negada, o usuário ainda pode buscar uma cidade manualmente.

23. Como rodar o projeto

Passo 1: clonar o repositório.

```
git clone https://github.com/WagnerFO/Open_Weather_API.git
```

Passo 2: entrar na pasta.

```
cd Open_Weather_API
```

Passo 3: criar o arquivo .env.

```
OPENWEATHER_API_KEY=SUA_CHAVE_AQUI
```

Passo 4: instalar dependências.

```
flutter pub get
```

Passo 5: rodar o app.

```
flutter run
```

24. Testes e qualidade

Durante o desenvolvimento foram usados:

```
flutter analyze  
flutter test
```

O flutter analyze verifica problemas de código, imports, tipos e boas práticas.

O flutter test executa os testes automatizados do projeto.

25. Desafios encontrados

Alguns desafios técnicos do projeto:

- configurar corretamente emulador Android;
- lidar com falta de espaço no emulador;
- proteger a chave da API usando .env;
- tratar permissão de localização;
- organizar estado global com Riverpod;
- evitar overflow em layouts com textos longos;
- integrar mapa com camadas de tiles.

26. Soluções aplicadas

Para o problema de espaço no emulador, foi criado um novo dispositivo virtual.

Para o overflow nos cards, foram usados Expanded, Flexible, FittedBox e TextOverflow.ellipsis.

Para o histórico, foi criado um provider específico que controla a lista das cidades recentes.

Para o mapa, foi usada uma solução leve baseada em tiles.

27. Pontos fortes do projeto

- Integração real com API externa.
- Arquitetura organizada em camadas.
- Uso de estado reativo com Riverpod.
- Interface com múltiplas telas.
- Mapa interativo com dados climáticos.
- Separação entre regra de negócio e apresentação.
- Projeto executável por outro usuário com documentação.

28. Limitações atuais

O histórico ainda não é persistido em armazenamento local.

As camadas do mapa dependem da disponibilidade dos servidores de tiles.

O app depende de uma chave válida da OpenWeather.

Alguns dados, como UV, podem ser simplificados na interface.

29. Melhorias futuras

Possíveis evoluções:

- salvar histórico localmente com SharedPreferences ou Hive;
- adicionar favoritos permanentes;
- criar modo escuro;
- melhorar tela de configurações;
- adicionar cache de clima;
- exibir detalhes de previsão diária;
- criar testes unitários para repository e providers;
- adicionar tratamento visual para falhas de internet.

30. Roteiro para apresentação oral

1. Apresentar o objetivo do app.

2. Mostrar a Home e a busca de cidades.
3. Explicar que a OpenWeather retorna dados por coordenadas.
4. Mostrar o histórico das últimas cidades buscadas.
5. Mostrar os alertas meteorológicos.
6. Mostrar o mapa e trocar entre chuva, nuvens, temperatura e vento.
7. Explicar rapidamente a arquitetura em camadas.
8. Mostrar o uso de Riverpod.
9. Explicar o arquivo .env e a proteção da chave da API.
10. Finalizar com desafios e melhorias futuras.

31. Conclusão

O projeto demonstra a construção de um aplicativo Flutter completo, com integração de API, estado global, navegação, geolocalização e mapa interativo.

Ele é um bom exemplo acadêmico porque une conceitos importantes de desenvolvimento mobile com um caso de uso real e fácil de entender: consulta climática.

Além disso, a organização em camadas permite que o projeto continue evoluindo com novas funcionalidades sem tornar o código difícil de manter.